

RYDO DEVELOPMENT DOCUMENTATION

Interactive Liquid Background Implementation

1. The Generated Prompt

Gemini CLI Prompt: Liquid Shader Integration

*"Please integrate the provided Three.js script and GLSL shaders to create a high-fidelity, interactive liquid background for the ****Home**** page of the ****RYDO**** application.*

*****1. Layout & Layering:*****

** Implement the `#container` div as a fixed, full-screen layer behind all other content (`z-index: -1`).*

** Ensure the RYDO navigation bar and the 'Go to Explorer' button remain visible and interactive above the animation.*

*****2. Shader & Animation Requirements:*****

** Use the Gaussian blur and noise-based fragment shader to achieve a smooth, metaball-style liquid effect.*

** The animation should be GPU-accelerated to maintain high frame rates on a ****Dell G15**** laptop.*

** Ensure the `u\_noise` uniform correctly pulls from the provided texture source to drive organic motion.*

*****3. User Interaction:*****

** The liquid distortion must follow the user's cursor smoothly using the established `pointermove` logic.*

** Include a subtle 'lag' effect (divisor) to make the liquid feel heavy and high-quality.*

*****4. Responsiveness:*****

** The background must automatically scale to fill the viewport on window resize without losing the liquid effect's proportion.*

** Maintain the render buffer scaling at 0.2 to balance visual quality with system performance."*

2. Implementation Code

Use the following code block for the Home page background logic:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/88/three.min.js"></script>
<script id="vertexShader" type="x-shader/x-vertex">
    void main() {
        gl_Position = vec4( position, 1.0 );
    }
</script>
<script id="fragmentShader" type="x-shader/x-fragment">
    uniform vec2 u_resolution;
    uniform vec2 u_mouse;
    uniform float u_time;
    uniform sampler2D u_noise;
    uniform sampler2D u_buffer;
    uniform bool u_renderpass;

    const float blurMultiplier = 0.95;
    const float circleSize = .25;
    const float blurStrength = .98;
    const float threshold = .5;
    const float scale = 4.;

    #define _fract true

    #define PI 3.141592653589793
    #define TAU 6.283185307179586

    vec2 hash2(vec2 p)
    {
        vec2 o = texture2D( u_noise, (p+0.5)/256.0, -100.0 ).xy;
        return o;
    }

    vec3 hsb2rgb( in vec3 c ){
        vec3 rgb = clamp(abs(mod(c.x*6.0+vec3(0.0,4.0,2.0),
                                                6.0)-3.0)-1.0,
                        0.0,
                        1.0 );
    }
</script>
```

```

    rgb = rgb*rgb*(3.0-2.0*rgb);
    return c.z * mix( vec3(1.0), rgb, c.y);
}

vec3 domain(vec2 z){
    return vec3(hsb2rgb(vec3(atan(z.y,z.x)/TAU,1.,1.)));
}

vec3 colour(vec2 z) {
    return domain(z);
}

#define pow2(x) (x * x)

const int samples = 8;
const float sigma = float(samples) * 0.25;

float gaussian(vec2 i) {
    return 1.0 / (2.0 * PI * pow2(sigma)) * exp(-((pow2(i.x) + pow2(i.y)) /
(2.0 * pow2(sigma))));
}

vec3 hash33(vec3 p){
    float n = sin(dot(p, vec3(7, 157, 113)));
    return fract(vec3(2097152, 262144, 32768)*n);
}

vec3 blur(sampler2D sp, vec2 uv, vec2 scale) {
    vec3 col = vec3(0.0);
    float accum = 0.0;
    float weight;
    vec2 offset;

    for (int x = -samples / 2; x < samples / 2; ++x) {
        for (int y = -samples / 2; y < samples / 2; ++y) {
            offset = vec2(x, y);
            weight = gaussian(offset);
            col += texture2D(sp, uv + scale * offset).rgb * weight;
            accum += weight;
        }
    }

    return col / accum;
}

void main() {
    vec2 uv = (gl_FragCoord.xy - 0.5 * u_resolution.xy) / u_resolution.y;
    uv *= scale;
    vec2 mouse = u_mouse * scale;

    vec2 ps = vec2(1.0) / u_resolution.xy;

```

```

vec2 sample = gl_FragCoord.xy / u_resolution.xy;
vec2 o = mouse*.2+vec2(.65, .5);
float d = .98;
sample = d * (sample - o);
sample += o;
sample += vec2(sin((u_time+uv.y * .5)*10.)*.001, -.00);

vec3 fragcolour;
vec4 tex;
if(u_renderpass) {
    tex = vec4(blur(u_buffer, sample, ps*blurStrength) * blurMultiplier, 1.);
    float df = length(mouse - uv);
    fragcolour = vec3( smoothstep( circleSize, 0., df ) );
} else {
    tex = texture2D(u_buffer, sample, 2.) * .98;
    tex = vec4(
        smoothstep(0.0, threshold - fwidth(tex.x), tex.x),
        smoothstep(0.2, threshold - fwidth(tex.y) + .2, tex.y),
        smoothstep(-0.05, threshold - fwidth(tex.z) - .2, tex.z),
        1.);
    vec3 n = hash33(vec3(uv, u_time*.1));
    tex.rgb += n * .2 - .1;
}
gl_FragColor = vec4(fragcolour,1.0);
gl_FragColor += tex;
}
</script>

```

```
<div id="container" touch-action="none"></div>
```

```
/* CSS */
```

```
body { margin: 0; padding: 0; }
```

```
#container { position: fixed; touch-action: none; }
```

```
/* JavaScript (Babel) */
```

```
let container;
```

```
let camera, scene, renderer;
```

```
let uniforms;
```

```
let divisor = 1 / 10;
```

```
let newmouse = { x: 0, y: 0 };
```

```
let loader = new THREE.TextureLoader();
```

```
let texture, rtTexture, rtTexture2;
```

```
loader.setCrossOrigin("anonymous");
```

```
loader.load(
```

```
    'https://s3-us-west-2.amazonaws.com/s.cdpn.io/982762/noise.png',
```

```
    function do_something_with_texture(tex) {
```

```
        texture = tex;
```

```
        texture.wrapS = THREE.RepeatWrapping;
```

```
        texture.wrapT = THREE.RepeatWrapping;
```

```
        texture.minFilter = THREE.LinearFilter;
```

```

        init();
        animate();
    }
};

function init() {
    container = document.getElementById( 'container' );
    camera = new THREE.Camera();
    camera.position.z = 1;
    scene = new THREE.Scene();
    var geometry = new THREE.PlaneBufferGeometry( 2, 2 );
    rtTexture = new THREE.WebGLRenderTarget(window.innerWidth * .2,
window.innerHeight * .2);
    rtTexture2 = new THREE.WebGLRenderTarget(window.innerWidth * .2,
window.innerHeight * .2);
    uniforms = {
        u_time: { type: "f", value: 1.0 },
        u_resolution: { type: "v2", value: new THREE.Vector2() },
        u_noise: { type: "t", value: texture },
        u_buffer: { type: "t", value: rtTexture.texture },
        u_mouse: { type: "v2", value: new THREE.Vector2() },
        u_renderpass: { type: 'b', value: false }
    };
    var material = new THREE.ShaderMaterial( {
        uniforms: uniforms,
        vertexShader: document.getElementById( 'vertexShader' ).textContent,
        fragmentShader: document.getElementById( 'fragmentShader' ).textContent
    } );
    material.extensions.derivatives = true;
    var mesh = new THREE.Mesh( geometry, material );
    scene.add( mesh );
    renderer = new THREE.WebGLRenderer();
    renderer.setPixelRatio( window.devicePixelRatio );
    container.appendChild( renderer.domElement );
    onWindowResize();
    window.addEventListener( 'resize', onWindowResize, false );
    document.addEventListener('pointermove', (e)=> {
        let ratio = window.innerHeight / window.innerWidth;
        newmouse.x = (e.pageX - window.innerWidth / 2) / window.innerWidth / ratio;
        newmouse.y = (e.pageY - window.innerHeight / 2) / window.innerHeight * -1;
        e.preventDefault();
    });
}

function onWindowResize( event ) {
    renderer.setSize( window.innerWidth, window.innerHeight );
    uniforms.u_resolution.value.x = renderer.domElement.width;
    uniforms.u_resolution.value.y = renderer.domElement.height;
    rtTexture = new THREE.WebGLRenderTarget(window.innerWidth * .2,
window.innerHeight * .2);

```

```

    rtTexture2 = new THREE.WebGLRenderTarget(window.innerWidth * .2,
window.innerHeight * .2);
}

function animate(delta) {
    requestAnimationFrame( animate );
    render(delta);
}

function renderTexture(delta) {
    let odims = uniforms.u_resolution.value.clone();
    uniforms.u_resolution.value.x = window.innerWidth * .2;
    uniforms.u_resolution.value.y = window.innerHeight * .2;
    uniforms.u_buffer.value = rtTexture2.texture;
    uniforms.u_renderpass.value = true;
    renderer.setRenderTarget(rtTexture);
    renderer.render( scene, camera, rtTexture, true );
    let buffer = rtTexture;
    rtTexture = rtTexture2;
    rtTexture2 = buffer;
    uniforms.u_buffer.value = rtTexture.texture;
    uniforms.u_resolution.value = odims;
    uniforms.u_renderpass.value = false;
}

function render(delta) {
    uniforms.u_mouse.value.x += ( newmouse.x - uniforms.u_mouse.value.x ) *
divisor;
    uniforms.u_mouse.value.y += ( newmouse.y - uniforms.u_mouse.value.y ) *
divisor;
    uniforms.u_time.value = delta * 0.0005;
    renderer.render( scene, camera );
    renderTexture();
}

```